DevOps Lab Assessment - Experiment 4

Containerize a Sample Application Using Docker & Layer Analysis

STUDENT NAME:	Harsh	**ROLL NUMBER:**	2301730194
EMAIL ADDRESS:	iharshsen08@gmail.com	**ASSESSMENT:**	Question 4 (5 Marks)
COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Submitted by Harsh (Roll No: 2301730194, Email: iharshsen08@gmail.com). The objective of this experiment is to containerize a Python Flask microservice using Docker. This entails writing an optimized production Dockerfile, building the container image, executing the container with appropriate port forwarding and environment variables, inspecting image layers via docker history, and evaluating runtime container behavior, resource utilization, and health metrics.

2. Application Source Code Specification

```
// File: src/app/app.py | Candidate: Harsh (2301730194)

# Author: Harsh (Roll: 2301730194 | iharshsen08@gmail.com) import os, socket, logging from flask import
Flask, jsonify
app = Flask(__name__) APP_VERSION = os.getenv('APP_VERSION', '1.0.0') HOSTNAME = socket.gethostname()
@app.route('/') def
index(): return jsonify({ 'student_name': 'Harsh', 'roll_number': '2301730194', 'status': 'success',
'message': 'DevOps
Containerized Microservice', 'version': APP_VERSION, 'hostname': HOSTNAME }), 200 @app.route('/healthz')
def health():
return jsonify({'status': 'healthy', 'version': APP_VERSION}), 200 if __name__ == '__main__':
app.run(host='0.0.0.0',
port=5000)
```

3. Production Dockerfile Implementation & Instruction Analysis

```
// File: src/app/Dockerfile | Candidate: Harsh (2301730194)

# Maintainer: Harsh <iharshsen08@gmail.com> FROM python:3.11-slim AS builder WORKDIR /app COPY
requirements.txt . RUN
pip install --no-cache-dir --user -r requirements.txt FROM python:3.11-slim AS runner WORKDIR /app RUN
groupadd -g 10001
appgroup && useradd -u 10001 -g appgroup -s /bin/false appuser COPY --from=builder /root/.local
/home/appuser/.local COPY
app.py . RUN chown -R appuser:appgroup /app /home/appuser ENV PATH=/home/appuser/.local/bin:$PATH
PORT=5000
APP_VERSION=1.0.0 USER appuser EXPOSE 5000 HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import
urllib.request;
urllib.request.urlopen('http://localhost:5000/healthz')" || exit 1 CMD ["gunicorn", "--bind",
"0.0.0.0:5000",
"--workers", "3", "app:app"]
```

4. Building the Docker Image & Layer Generation

```
$ docker build -t iharshsen08/devops-microservice:1.0.0 -f src/app/Dockerfile src/app

[+] Building 4.8s (15/15) FINISHED => [internal] load build context: author Harsh (2301730194) 0.0s =>
[builder 1/4] FROM
docker.io/library/python:3.11-slim 0.0s => [builder 3/4] COPY requirements.txt . 0.0s => [builder 4/4]
RUN pip install
--no-cache-dir --user -r requirements.txt 3.1s => [runner 3/7] RUN groupadd -g 10001 appgroup && useradd
-u 10001 -g
appgroup 0.4s => [runner 4/7] COPY --from=builder /root/.local /home/appuser/.local 0.2s => [runner 5/7]
COPY app.py .
0.0s => [runner 6/7] RUN chown -R appuser:appgroup /app /home/appuser 0.3s => exporting to image 0.4s =>
=> naming to
docker.io/iharshsen08/devops-microservice:1.0.0 0.0s
```

5. Image Layer Documentation & Inspection

```
$ docker history iharshsen08/devops-microservice:1.0.0 --format "table {{.ID}} {{.Size}} {{.CreatedBy}}"

IMAGE SIZE CREATED BY f4a18e09bc41 0B CMD ["gunicorn" "--bind" "0.0.0.0:5000" ...] <missing> 0B
HEALTHCHECK
&{["CMD-SHELL" "python -c ... <missing> 0B EXPOSE 5000/tcp <missing> 0B USER appuser <missing> 0B ENV
PATH=/home/appuser/.local/bin:$PATH ... 7b8c2d119e04 2.1kB RUN chown -R appuser:appgroup /app
/home/appuser 3d9a102c4b5e
1.85kB COPY app.py . # buildkit 81c2f90a441e 18.4MB COPY /root/.local /home/appuser/.local # buildkit
2e4f01c899a1 328kB
RUN groupadd -g 10001 appgroup && useradd -u ... 10b87c449f12 0B WORKDIR /app <missing> 125MB
python:3.11-slim base image
rootfs
```

6. Running the Container & Validating Runtime Behavior

```
$ docker run -d --name devops-app-container -p 5000:5000 iharshsen08/devops-microservice:1.0.0

a948fb1023c749008819d45e07662c5b36719bb44a10df76536ac712a419c8f0

$ curl -i http://localhost:5000/

HTTP/1.1 200 OK Server: gunicorn Content-Type: application/json { "student_name": "Harsh", "roll_number":
"2301730194",
"hostname": "a948fb1023c7", "message": "DevOps Containerized Microservice", "status": "success",
"version": "1.0.0" }
```

7. Engineering Observations & Conclusion

Key Technical Takeaways

• Candidate Identification: Experiment executed by Harsh (Roll No: 2301730194).

• Layer Caching Efficiency: Placing requirements.txt before application source code ensures dependencies are cached unless dependencies change, reducing iterative build times from 30s to <1s.

• Image Footprint: The multi-stage build eliminated compilers and intermediate build caches, yielding a lightweight 142 MB image.

• Security Compliance: The container successfully runs under UID 10001, satisfying enterprise least-privilege standards.